

Exercices Mathématiques & Théoriques

Programme Data Science – Phases 0 à 4

Hakim – Support pédagogique

Objectif : dérouler « à la main » les algorithmes vus en cours (calculs de coût, itérations de descente de gradient, calculs de distance, d'impureté, d'algorithmes de clustering, etc.), du niveau débutant à avancé, phase par phase. Chaque exercice est accompagné d'un corrigé détaillé, sur le modèle des TD de référence (calcul explicite, étape par étape).

Contents

1	Mise à niveau – Python, algèbre linéaire, statistiques	2
1.0.1	Vecteurs	2
1.0.2	Statistiques descriptives	2
1.0.3	Produit matriciel et commutativité	2
1.0.4	Normalisation min-max et standardisation	3
1.0.5	Inversion de matrice et système linéaire	3
1.0.6	Complexité et vectorisation	3
2	Régression & fondamentaux	4
2.0.1	Régression linéaire simple par moindres carrés	4
2.0.2	Calcul de l'erreur MSE	4
2.0.3	Une itération de descente de gradient	4
2.0.4	Régression logistique – probabilité et coût	4
2.0.5	Descente de gradient sur deux itérations (convergence)	5
2.0.6	Régularisation Ridge	5
3	Algorithmes avancés – k-NN, arbres, ensembles	6
3.0.1	k-NN à la main	6
3.0.2	Entropie et indice de Gini	6
3.0.3	Gain d'information sur un split	6
3.0.4	Bagging et réduction de variance	6
3.0.5	Une itération d'AdaBoost	7
3.0.6	Gradient Boosting – résidus	7
4	Non supervisé & SVM	8
4.0.1	Marge géométrique d'un SVM	8
4.0.2	K-Means : une itération en 1D	8
4.0.3	Matrice de covariance et PCA	8
4.0.4	Astuce du noyau (kernel trick)	9
4.0.5	Vecteur propre principal et projection (PCA)	9
4.0.6	Hinge loss et effet de C	9
5	Projet de synthèse – évaluation et sélection de modèle	11
5.0.1	Métriques à partir d'une matrice de confusion	11
5.0.2	Stabilité par validation croisée	11
5.0.3	Combinatoire d'une recherche sur grille	11
5.0.4	Diagnostic biais-variance	11
5.0.5	Méthodologie et fuite de données (data leakage)	12

Phase 1 Mise à niveau – Python, algèbre linéaire, statistiques

Séances 1 à 9 – prérequis : aucun.

Niveau débutant

Exercice 1.0.1 Vecteurs

Soit $u = (2, -1, 3)$ et $v = (1, 4, 0)$.

- Calculer $u + v$.
- Calculer le produit scalaire $u \cdot v$.
- Calculer la norme euclidienne $\|u\|$.

Corrigé

- $u + v = (3, 3, 3)$.
- $u \cdot v = 2 \times 1 + (-1) \times 4 + 3 \times 0 = 2 - 4 + 0 = -2$.
- $\|u\| = \sqrt{2^2 + (-1)^2 + 3^2} = \sqrt{14} \approx 3.74$.

Exercice 1.0.2 Statistiques descriptives

Considérer l'échantillon d'âges suivant : 18, 22, 25, 30, 35.

- Calculer la moyenne $\bar{x}$.
- Calculer la variance σ^2 et l'écart-type σ .

Corrigé

$$\bar{x} = \frac{18 + 22 + 25 + 30 + 35}{5} = \frac{130}{5} = 26.$$

Écarts à la moyenne : -8, -4, -1, 4, 9 ; carrés : 64, 16, 1, 16, 81, somme = 178.

$$\sigma^2 = \frac{178}{5} = 35.6 \Rightarrow \sigma = \sqrt{35.6} \approx 5.97.$$

Niveau intermédiaire

Exercice 1.0.3 Produit matriciel et commutativité

Soit $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ et $B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$.

- Calculer AB et BA .
- Le produit matriciel est-il commutatif ? Conclure.

Corrigé

$$AB = \begin{pmatrix} 1 \times 0 + 2 \times 1 & 1 \times 1 + 2 \times 0 \\ 3 \times 0 + 4 \times 1 & 3 \times 1 + 4 \times 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}$$

$$BA = \begin{pmatrix} 0 \times 1 + 1 \times 3 & 0 \times 2 + 1 \times 4 \\ 1 \times 1 + 0 \times 3 & 1 \times 2 + 0 \times 4 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}$$

$AB \neq BA$: le produit matriciel n'est **pas commutatif** en général – c'est pourquoi l'ordre des opérations (ex. $X\theta$ vs θX) compte en ML.

Exercice 1.0.4 Normalisation min-max et standardisation

Soit $x = [10, 20, 30, 40, 50]$.

- Calculer x normalisé (min-max, entre 0 et 1).
- Calculer x standardisé (z-score).

Corrigé

- a) $x_{\min} = 10, x_{\max} = 50 : x'_i = \frac{x_i - 10}{40}$ donne $[0, 0.25, 0.5, 0.75, 1]$.
- b) $\bar{x} = 30$. Écarts : $-20, -10, 0, 10, 20$; carrés : $400, 100, 0, 100, 400$, somme = 1000. $\sigma^2 = 200 \Rightarrow \sigma \approx 14.14$. $z_i = \frac{x_i - 30}{14.14}$ donne $[-1.41, -0.71, 0, 0.71, 1.41]$.

Niveau avancé**Exercice 1.0.5 Inversion de matrice et système linéaire**

Résoudre $Ax = b$ avec $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$, $b = \begin{pmatrix} 5 \\ 10 \end{pmatrix}$, en calculant A^{-1} .

Corrigé

$$\det(A) = 2 \times 3 - 1 \times 1 = 5.$$

$$A^{-1} = \frac{1}{5} \begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}.$$

$$x = A^{-1}b = \frac{1}{5} \begin{pmatrix} 3 \times 5 - 1 \times 10 \\ -1 \times 5 + 2 \times 10 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 5 \\ 15 \end{pmatrix} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}.$$

Vérification : $2(1) + 1(3) = 5 \checkmark$; $1(1) + 3(3) = 10 \checkmark$.

Exercice 1.0.6 Complexité et vectorisation

On veut calculer $\sum_i x_i^2$ pour un vecteur x de taille n .

- Donner la complexité d'une boucle `for` classique.
- Donner la complexité de l'opération vectorisée `np.sum(x**2)`.
- Les deux ont-elles la même complexité asymptotique ? Pourquoi NumPy est-il malgré tout bien plus rapide en pratique ?

Corrigé

- a) et b) : les deux sont en $O(n)$ (un passage sur les n éléments).
- c) Même complexité asymptotique, mais NumPy exécute la boucle en code C compilé, utilise des instructions vectorielles (SIMD) et évite l'overhead de l'interpréteur Python à chaque itération : la constante cachée devant le $O(n)$ est bien plus petite.

Phase 2 Régression & fondamentaux

Séances 4 à 7 – prérequis : Phase 0, NumPy, dérivées de base.

Niveau débutant

Exercice 2.0.1 Régression linéaire simple par moindres carrés

Soit les points $(1, 2), (2, 3), (3, 5)$. Calculer a et b du modèle $y = ax + b$ par la méthode des moindres carrés :

$$a = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad b = \bar{y} - a\bar{x}.$$

Corrigé

$\bar{x} = 2, \bar{y} = \frac{2+3+5}{3} = 3.33$.
 $\sum (x_i - \bar{x})(y_i - \bar{y}) = (-1)(-1.33) + (0)(-0.33) + (1)(1.67) = 1.33 + 0 + 1.67 = 3$.
 $\sum (x_i - \bar{x})^2 = 1 + 0 + 1 = 2$.
 $a = 3/2 = 1.5, \quad b = 3.33 - 1.5 \times 2 = 0.33$.
 Modèle : $y = 1.5x + 0.33$.

Exercice 2.0.2 Calcul de l'erreur MSE

Un modèle prédit $\hat{y} = [2.5, 3, 4.5]$ pour $y = [2, 3, 5]$. Calculer le $MSE = \frac{1}{m} \sum (\hat{y}_i - y_i)^2$.

Corrigé

Erreurs : 0.5, 0, -0.5. Carrés : 0.25, 0, 0.25. Somme = 0.5.
 $MSE = 0.5/3 \approx 0.167$.

Niveau intermédiaire

Exercice 2.0.3 Une itération de descente de gradient

Modèle sans biais $\hat{y} = \theta x$. Données $X = [1, 2, 3], Y = [2, 3, 5], \theta_0 = 0, \alpha = 0.1$.

- Calculer les prédictions et l'erreur $(\hat{y} - y)$.
- Calculer le gradient $\nabla J(\theta) = \frac{2}{m} \sum x_i(\hat{y}_i - y_i)$.
- Mettre à jour θ : $\theta \leftarrow \theta - \alpha \nabla J(\theta)$.

Corrigé

- $\hat{y} = [0, 0, 0]$, erreur = $[-2, -3, -5]$.
- $\nabla J(\theta) = \frac{2}{3}(1(-2) + 2(-3) + 3(-5)) = \frac{2}{3}(-23) = -15.33$.
- $\theta \leftarrow 0 - 0.1 \times (-15.33) = 1.533$.

Exercice 2.0.4 Régression logistique – probabilité et coût

Pour une observation, le score linéaire est $z = 1.2$ et l'étiquette réelle $y = 1$.

- Calculer $\sigma(z) = \frac{1}{1 + e^{-z}}$.
- Donner la classe prédite (seuil 0.5).
- Calculer la perte log-loss $-[y \log(p) + (1 - y) \log(1 - p)]$.

Corrigé

- a) $\sigma(1.2) = \frac{1}{1 + e^{-1.2}} = \frac{1}{1 + 0.301} \approx 0.769$.
 b) $0.769 > 0.5 \Rightarrow$ classe prédite = 1.
 c) $y = 1 \Rightarrow$ perte = $-\log(0.769) \approx 0.263$.

Niveau avancé**Exercice 2.0.5 Descente de gradient sur deux itérations (convergence)**

Modèle $\hat{y} = \theta_0 + \theta_1 x$. Données $X = [1, 2, 3]$, $Y = [2, 3, 5]$, $\theta = [0, 0]$, $\alpha = 0.1$. Dérouler **deux itérations** de descente de gradient batch et calculer le MSE avant/après.

Corrigé

Itération 1 : $\hat{y} = [0, 0, 0]$, erreur = $[-2, -3, -5]$.

$$\nabla_{\theta_0} = \frac{2}{3}(-2 - 3 - 5) = -6.667, \quad \nabla_{\theta_1} = \frac{2}{3}(1(-2) + 2(-3) + 3(-5)) = -15.333.$$

$$\theta \leftarrow [0 - 0.1(-6.667), 0 - 0.1(-15.333)] = [0.667, 1.533].$$

Itération 2 : $\hat{y} = [0.667 + 1.533(1), 0.667 + 1.533(2), 0.667 + 1.533(3)] = [2.20, 3.73, 5.27]$.
 erreur = $[0.20, 0.73, 0.27]$.

$$\nabla_{\theta_0} = \frac{2}{3}(0.20 + 0.73 + 0.27) = 0.80, \quad \nabla_{\theta_1} = \frac{2}{3}(0.20 + 1.47 + 0.80) = 1.644.$$

$$\theta \leftarrow [0.667 - 0.1(0.80), 1.533 - 0.1(1.644)] = [0.587, 1.368].$$

MSE : au départ ($\theta = 0$) : $\frac{4 + 9 + 25}{3} = 12.67$. Après l'itération 1 : $\frac{0.04 + 0.53 + 0.07}{3} \approx 0.216$.

Le coût chute très rapidement : la descente de gradient converge bien avec $\alpha = 0.1$ sur ce petit exemple.

Exercice 2.0.6 Régularisation Ridge

La fonction de coût Ridge est $J(\theta) = MSE(\theta) + \lambda \sum_j \theta_j^2$.

- a) Donner le gradient de J par rapport à θ_j (pour $j \neq 0$, sans régulariser le biais).
 b) Avec le gradient MSE $\nabla_{\theta_1} = 1.644$ (exercice précédent, $\theta_1 = 1.533$) et $\lambda = 0.1$, calculer le gradient régularisé puis la mise à jour avec $\alpha = 0.1$.
 c) Expliquer qualitativement l'effet d'un λ trop grand.

Corrigé

a) $\nabla_{\theta_j} J = \nabla_{\theta_j} MSE + 2\lambda\theta_j$.

b) $\nabla_{\theta_1} J = 1.644 + 2(0.1)(1.533) = 1.644 + 0.307 = 1.951$.

$\theta_1 \leftarrow 1.533 - 0.1 \times 1.951 = 1.338$ (légèrement plus petit qu'en régression non régularisée : 1.368).

c) Un λ trop grand force les coefficients vers 0 : le modèle sous-apprend (biais élevé, sous-ajustement).

Phase 3 Algorithmes avancés – k-NN, arbres, ensembles

Séances 8 à 11 – prérequis : Phases 0-1, classification, évaluation.

Niveau débutant

Exercice 3.0.1 k-NN à la main

Points d'entraînement : $A_1(1, 1)$ classe 0, $A_2(2, 1)$ classe 0, $A_3(4, 3)$ classe 1, $A_4(5, 4)$ classe 1, $A_5(1, 0)$ classe 0. Nouveau point $P(3, 2)$, $k = 3$, distance euclidienne.

Corrigé

$d(P, A_1) = \sqrt{4 + 1} = 2.24$; $d(P, A_2) = \sqrt{1 + 1} = 1.41$; $d(P, A_3) = \sqrt{1 + 1} = 1.41$; $d(P, A_4) = \sqrt{4 + 4} = 2.83$; $d(P, A_5) = \sqrt{4 + 4} = 2.83$.

3 plus proches voisins : $A_2(0, d=1.41)$, $A_3(1, d=1.41)$, $A_1(0, d=2.24)$.

Vote majoritaire : classes $\{0, 1, 0\} \Rightarrow$ **classe 0**.

Exercice 3.0.2 Entropie et indice de Gini

Un nœud contient 6 exemples de classe A et 4 de classe B. Calculer l'entropie et l'indice de Gini de ce nœud.

Corrigé

$p_A = 0.6$, $p_B = 0.4$.

Entropie = $-0.6 \log_2(0.6) - 0.4 \log_2(0.4) = 0.442 + 0.529 = 0.971$.

Gini = $1 - (0.6^2 + 0.4^2) = 1 - 0.52 = 0.48$.

Niveau intermédiaire

Exercice 3.0.3 Gain d'information sur un split

8 instances, attribut classe **Jouer**. Attribut **Météo** : Soleil (5 instances : 3 Oui, 2 Non), Pluie (3 instances : 1 Oui, 2 Non). Calculer $Info(D)$, $Info_{Mto}(D)$ et $Gain(Mto)$.

Corrigé

Global : 4 Oui, 4 Non $\Rightarrow Info(D) = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$.

$I(\text{Soleil}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$.

$I(\text{Pluie}) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.918$.

$Info_{Mto}(D) = \frac{5}{8}(0.971) + \frac{3}{8}(0.918) = 0.607 + 0.344 = 0.951$.

$Gain(Mto) = 1 - 0.951 = 0.049$ (gain faible : cet attribut sépare peu les classes).

Exercice 3.0.4 Bagging et réduction de variance

5 arbres bootstrap donnent les probabilités suivantes pour la classe 1 sur un même point test : $[0.60, 0.55, 0.70, 0.40, 0.65]$.

- Calculer la prédiction agrégée (moyenne) et la décision finale (seuil 0.5).
- Un seul arbre isolé donnant 0.40 prédirait la classe 0. Expliquer pourquoi l'agrégation change la décision et pourquoi c'est généralement souhaitable.

Corrigé

a) Moyenne = $\frac{0.60 + 0.55 + 0.70 + 0.40 + 0.65}{5} = \frac{2.90}{5} = 0.58 \Rightarrow$ **classe 1**.

b) Un arbre seul est instable (forte variance) et peut se tromper à cause du bruit de son échantillon bootstrap. En moyennant plusieurs arbres indépendants, les erreurs aléatoires se compensent : la variance de la prédiction diminue sans augmenter le biais – c'est le principe du bagging.

Niveau avancé

Exercice 3.0.5 Une itération d'AdaBoost

5 points, poids initiaux uniformes $w_i = 1/5$. Un classifieur faible se trompe sur 1 point.

- Calculer l'erreur pondérée ε .
- Calculer le coefficient $\alpha = \frac{1}{2} \ln \frac{1 - \varepsilon}{\varepsilon}$.
- Mettre à jour et normaliser les poids ($w \leftarrow w e^{\pm\alpha}$).

Corrigé

a) $\varepsilon = 1/5 = 0.2$.
 b) $\alpha = 0.5 \ln(0.8/0.2) = 0.5 \ln 4 = 0.693$.
 c) Point mal classé : $w \leftarrow 0.2 e^{0.693} = 0.2 \times 2 = 0.4$.
 Points bien classés (4) : $w \leftarrow 0.2 e^{-0.693} = 0.2 \times 0.5 = 0.1$ chacun.
 Somme avant normalisation = $0.4 + 4(0.1) = 0.8$.
 Poids normalisés : mal classé $\rightarrow 0.4/0.8 = 0.5$; bien classés $\rightarrow 0.1/0.8 = 0.125$ chacun (somme = 1 ✓).
 Le point mal classé pèse désormais bien plus lourd pour le classifieur suivant.

Exercice 3.0.6 Gradient Boosting – résidus

$y = [3, 5, 2, 8]$. Prédiction initiale $F_0 = \bar{y}$.

- Calculer F_0 et les résidus $r_i = y_i - F_0$.
- Expliquer ce que représentent ces résidus pour l'arbre h_1 suivant, et le rôle du taux d'apprentissage η dans $F_1 = F_0 + \eta h_1(x)$.

Corrigé

a) $F_0 = \frac{3 + 5 + 2 + 8}{4} = 4.5$. Résidus = $[-1.5, 0.5, -2.5, 3.5]$.
 b) Chaque nouvel arbre du Gradient Boosting apprend à prédire les résidus (l'erreur restante) du modèle courant, pas y directement. η (petit, ex. 0.1) freine la correction apportée par chaque arbre pour éviter le surapprentissage et rendre la convergence plus stable et progressive.

Phase 4 Non supervisé & SVM

Séances 12 à 14 – prérequis : Phases 0-2.

Niveau débutant

Exercice 4.0.1 Marge géométrique d'un SVM

Hyperplan $w \cdot x + b = 0$ avec $w = (3, 4)$, $b = -5$. Point $x = (2, 3)$. Calculer la distance signée $\frac{|w \cdot x + b|}{\|w\|}$.

Corrigé

$w \cdot x = 3(2) + 4(3) = 18$, $w \cdot x + b = 18 - 5 = 13$.
 $\|w\| = \sqrt{3^2 + 4^2} = 5$.
 Distance = $13/5 = 2.6$.

Exercice 4.0.2 K-Means : une itération en 1D

Points $[2, 4, 10, 12, 3, 20]$, $k = 2$, centroïdes initiaux $c_1 = 2$, $c_2 = 10$ (distance absolue).

- Affecter chaque point au centroïde le plus proche.
- Recalculer les nouveaux centroïdes.
- Refaire une itération : le résultat change-t-il ?

Corrigé

a) $2 \rightarrow c_1$ (0 vs 8) ; $4 \rightarrow c_1$ (2 vs 6) ; $10 \rightarrow c_2$ (8 vs 0) ; $12 \rightarrow c_2$ (10 vs 2) ; $3 \rightarrow c_1$ (1 vs 7) ; $20 \rightarrow c_2$ (18 vs 10).
 $C_1 = \{2, 4, 3\}$, $C_2 = \{10, 12, 20\}$.
 b) Nouveaux centroïdes : $c_1 = (2 + 4 + 3)/3 = 3$; $c_2 = (10 + 12 + 20)/3 = 14$.
 c) Avec $c_1 = 3$, $c_2 = 14$: $2 \rightarrow C_1$, $4 \rightarrow C_1$, $10 \rightarrow C_2$, $12 \rightarrow C_2$, $3 \rightarrow C_1$, $20 \rightarrow C_2$ – **mêmes clusters**. L'algorithme a convergé : $C_1 = \{2, 3, 4\}$ (moyenne 3), $C_2 = \{10, 12, 20\}$ (moyenne 14).

Niveau intermédiaire

Exercice 4.0.3 Matrice de covariance et PCA

Dataset centré 2D : $(1, 2), (3, 3), (5, 6)$.

- Calculer $\bar{x}, \bar{y}$ puis centrer les points.
- Calculer la matrice de covariance $\begin{pmatrix} \text{var}(x) & \text{cov}(x, y) \\ \text{cov}(x, y) & \text{var}(y) \end{pmatrix}$ (diviser par $n = 3$).

Corrigé

a) $\bar{x} = 3$, $\bar{y} = 3.67$. Points centrés : $(-2, -1.67), (0, -0.67), (2, 2.33)$.
 b) $\text{var}(x) = \frac{4 + 0 + 4}{3} = 2.67$.
 $\text{var}(y) = \frac{2.79 + 0.45 + 5.43}{3} \approx 2.89$.
 $\text{cov}(x, y) = \frac{(-2)(-1.67) + 0(-0.67) + 2(2.33)}{3} = \frac{3.34 + 0 + 4.66}{3} \approx 2.67$.

$Cov = \begin{pmatrix} 2.67 & 2.67 \\ 2.67 & 2.89 \end{pmatrix}$ – forte covariance positive : x et y varient ensemble, une seule direction principale devrait capter l'essentiel de la variance (voir exercice avancé).

Exercice 4.0.4 Astuce du noyau (kernel trick)

Les points $x = -2$ (classe -1), $x = -1$ (classe $+1$), $x = 1$ (classe $+1$), $x = 2$ (classe -1) ne sont pas linéairement séparables en 1D.

- Proposer une transformation $\varphi(x)$ simple qui rend les données séparables.
- Calculer $\varphi(x)$ pour chaque point et donner un seuil de séparation.

Corrigé

a) $\varphi(x) = x^2$ (noyau polynomial de degré 2).
 b) $\varphi(-2) = 4$ (classe -1), $\varphi(-1) = 1$ (classe $+1$), $\varphi(1) = 1$ (classe $+1$), $\varphi(2) = 4$ (classe -1).
 Dans ce nouvel espace, un simple seuil comme $\varphi(x) < 2.5 \Rightarrow +1$, $\varphi(x) \geq 2.5 \Rightarrow -1$ sépare parfaitement les classes : les données, non séparables linéairement en 1D, le deviennent après projection.

Niveau avancé

Exercice 4.0.5 Vecteur propre principal et projection (PCA)

En reprenant $Cov = \begin{pmatrix} 2.67 & 2.67 \\ 2.67 & 2.89 \end{pmatrix}$ de l'exercice précédent :

- Calculer les valeurs propres λ_1, λ_2 (résoudre $\det(Cov - \lambda I) = 0$).
- Quelle proportion de la variance totale la première composante explique-t-elle ?

Corrigé

a) $(2.67 - \lambda)(2.89 - \lambda) - 2.67^2 = 0 \Rightarrow \lambda^2 - 5.56\lambda + 0.587 = 0$.
 $\lambda = \frac{5.56 \pm \sqrt{5.56^2 - 4(0.587)}}{2} = \frac{5.56 \pm \sqrt{28.58}}{2} = \frac{5.56 \pm 5.35}{2}$.
 $\lambda_1 \approx 5.45$, $\lambda_2 \approx 0.11$.
 b) Proportion = $\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{5.45}{5.56} \approx 98\%$.
 La première composante principale capte à elle seule presque toute l'information : on peut réduire les données à 1 dimension avec très peu de perte.

Exercice 4.0.6 Hinge loss et effet de C

Pour 3 points, $y \cdot f(x)$ vaut respectivement 1.5, 0.5, -0.2 . $\|w\|^2 = 4$, $C = 1$.

- Calculer la hinge loss $\max(0, 1 - yf(x))$ de chaque point.
- Calculer le coût total $\frac{1}{2}\|w\|^2 + C \sum_i \text{hinge}_i$.
- Expliquer qualitativement ce qui se passe si on augmente fortement C .

Corrigé

a) Point 1 : $\max(0, 1 - 1.5) = 0$ (bien classé, hors marge). Point 2 : $\max(0, 1 - 0.5) = 0.5$. Point 3 : $\max(0, 1 - (-0.2)) = 1.2$.
 b) Coût = $0.5 \times 4 + 1 \times (0 + 0.5 + 1.2) = 2 + 1.7 = 3.7$.

c) Un C très grand pénalise fortement les erreurs : le modèle cherche à tout classer correctement, quitte à réduire la marge et à sur-apprendre (moins de tolérance aux points mal placés, marge dure/soft margin plus stricte).

Phase 5 Projet de synthèse – évaluation et sélection de modèle

Séances 15 & 16 – synthèse de toutes les phases.

Niveau débutant

Exercice 5.0.1 Métriques à partir d'une matrice de confusion

$TP = 40$, $FP = 10$, $FN = 5$, $TN = 45$. Calculer exactitude, précision, rappel et F1-score.

Corrigé

$$\begin{aligned}\text{Exactitude} &= \frac{40 + 45}{100} = 0.85. \\ \text{Précision} &= \frac{40}{40 + 10} = 0.80. \\ \text{Rappel} &= \frac{40}{40 + 5} = 0.889. \\ \text{F1} &= 2 \times \frac{0.80 \times 0.889}{0.80 + 0.889} \approx 0.842.\end{aligned}$$

Niveau intermédiaire

Exercice 5.0.2 Stabilité par validation croisée

5 scores de CV : $[0.82, 0.85, 0.79, 0.88, 0.84]$. Calculer la moyenne et l'écart-type, puis commenter la stabilité du modèle.

Corrigé

Moyenne $= 4.18/5 = 0.836$.
 Écarts : $-0.016, 0.014, -0.046, 0.044, 0.004$; carrés (approx.) : $0.00026, 0.00020, 0.00212, 0.00194, 0.00002$, somme ≈ 0.00452 .
 $\sigma^2 = 0.00452/5 = 0.000904 \Rightarrow \sigma \approx 0.030$.
 Un écart-type faible (3 points) autour d'une moyenne de 83.6% indique un modèle **stable** d'un pli à l'autre.

Exercice 5.0.3 Combinatoire d'une recherche sur grille

GridSearchCV avec $n_estimators=[100, 200, 400]$, $max_depth=[4, 8, \text{None}]$, $cv=5$. Combien de modèles au total sont réellement entraînés ?

Corrigé

Nombre de combinaisons d'hyperparamètres $= 3 \times 3 = 9$.
 Chaque combinaison est évaluée sur les 5 plis $\Rightarrow 9 \times 5 = 45$ modèles entraînés.

Niveau avancé

Exercice 5.0.4 Diagnostic biais-variance

Un modèle obtient un score d'entraînement de 0.98 et un score de validation de 0.75.

- Diagnostiquer la situation.
- Proposer 3 solutions concrètes.

Corrigé

- a) Grand écart entraînement/validation $\Rightarrow$ **surapprentissage** (variance élevée).
- b) Solutions possibles : (1) ajouter de la régularisation (L1/L2, limiter `max_depth`) ; (2) collecter davantage de données d'entraînement ; (3) réduire la complexité du modèle ou faire de la sélection/réduction de variables (ex. PCA).

Exercice 5.0.5 Méthodologie et fuite de données (data leakage)

Un modèle baseline a un log-loss de 0.65. Après `GridSearchCV(cv=5)` sur 60 combinaisons, le meilleur modèle atteint 0.42. L'équipe envisage d'ajuster le seuil de décision directement sur le jeu de test pour améliorer encore le F1-score.

- a) Pourquoi cette pratique est-elle risquée ?
- b) Proposer la méthodologie correcte.

Corrigé

- a) Ajuster n'importe quel paramètre (y compris un seuil de décision) sur le jeu de test revient à laisser le test « fuiter » dans le processus d'entraînement/optimisation : le score obtenu devient optimiste et ne reflète plus la performance réelle sur des données jamais vues.
- b) Découper les données en train / validation / test (ou faire une validation croisée imbriquée – *nested CV*) : optimiser les hyperparamètres *et* le seuil de décision uniquement sur train/validation, puis évaluer une seule fois, à la toute fin, sur le jeu de test intact.